

ASSIGNMENT-13.2

HALLTICKET:2303A51606

BATCH-03

TASK-01

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

- Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

- Ask AI to locate repeated computations within the program.
- Encapsulate repeated logic into reusable functions.
- Verify that program output remains unchanged after refactoring.
- Add proper docstrings to all defined functions.

Sample Code:

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

```
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

PROMPT:

Refactor the Python code by identifying repeated calculations and creating a reusable method to compute area and perimeter without changing the output.

ORIGINAL CODE:

```
lab_14_5.py > ...
1  length = 8
2  width = 4
3  print("Area:", length * width)
4  print("Perimeter:", 2 * (length + width))
5
6  length = 12
7  width = 6
8  print("Area:", length * width)
9  print("Perimeter:", 2 * (length + width))
```

OUTPUT:

```
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

REFACTORED CODE:

```
lab_14_5.py > ...
1  def rectangle_details(length, width):
2      """
3      Calculate and print the area and perimeter of a rectangle.
4
5      Parameters:
6      length (int/float): Length of the rectangle
7      width (int/float): Width of the rectangle
8      """
9
10     area = length * width
11     perimeter = 2 * (length + width)
12
13     print("Area:", area)
14     print("Perimeter:", perimeter)
15
16
17 # First rectangle
18 rectangle_details(8, 4)
19
20 # Second rectangle
21 rectangle_details(12, 6)
```

OUTPUT:

```
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

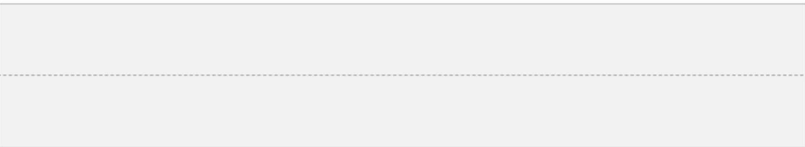
JUSTIFICATION:

Eliminating Repetitive Logic

Refactoring removes duplicate calculations by placing them inside a reusable method. This improves code maintainability and makes it easier to reuse the same logic for different inputs.

TASK2:

Task Description -2 (Refactoring – Modularizing Inline Calculations) • Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions. Instructions: • Identify repeated calculation patterns in the code. • Convert them into reusable functions. • Ensure readability and proper documentation. Sample Code: <pre>amount = 1000 discount = amount * 0.10 final_amount = amount - discount print("Final Amount:", final_amount)</pre>	
---	--



<pre>amount = 2000 discount = amount * 0.10 final_amount = amount - discount print("Final Amount:", final_amount)</pre> Expected Output: A modular Python program that uses a reusable function to compute final amounts for different inputs.	
---	--

PROMPT:

Extract the repeated discount calculation into a reusable Python method that computes the final amount after a 10% discount.

ORIGINAL CODE:

```
lab_14_5.py > ...
1  amount = 1000
2  discount = amount * 0.10
3  final_amount = amount - discount
4  print("Final Amount:", final_amount)
5
6  amount = 2000
7  discount = amount * 0.10
8  final_amount = amount - discount
9  print("Final Amount:", final_amount)
```

OUTPUT:

```
Final Amount: 900.0
Final Amount: 1800.0
```

REFACTORED CODE:

```
lab_14_5.py > calculate_final_amount
1  def calculate_final_amount(amount):
2      """
3      Calculate the final amount after applying a 10% discount.
4      Parameters:
5      amount (int/float): Original amount
6      Returns:
7      float: Final amount after discount
8      """
9      discount = amount * 0.10
10     final_amount = amount - discount
11     return final_amount
12     print("Final Amount:", calculate_final_amount(1000))
13     print("Final Amount:", calculate_final_amount(2000))
```

OUTPUT:

```
Final Amount: 900.0
Final Amount: 1800.0
```

JUSTIFICATION:

Modularizing Inline Calculations

Creating a separate method for the discount calculation reduces code repetition and improves modularity. It also makes the program easier to update if the discount logic changes.

TASK3:

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

- Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Instructions:

- Use AI to recommend more efficient data structures or logic simplifications.
- Replace inefficient constructs while preserving program behavior.
- Compare execution efficiency before and after refactoring.

Sample Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
```

```
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
```

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

PROMPT:

Optimize the Python code that uses nested loops for searching by using a more efficient data structure while keeping the same output.

ORIGINAL CODE:

```

lab_14_5.py > ...
1  students = ["Anil", "Bhavya", "Charan", "Divya"]
2  check_list = ["Charan", "Esha", "Bhavya"]
3
4  for name in check_list:
5      exists = False
6      for student in students:
7          if name == student:
8              exists = True
9
10     if exists:
11         print(name, "found")
12     else:
13         print(name, "not found")

```

OUTPUT:

```

Charan found
Esha not found
Bhavya found

```

REFACTORED CODE:

```

lab_14_5.py > ...
1  students = ["Anil", "Bhavya", "Charan", "Divya"]
2  check_list = ["Charan", "Esha", "Bhavya"]
3
4  student_set = set(students) # faster lookup
5
6  for name in check_list:
7      if name in student_set:
8          print(name, "found")
9      else:
10         print(name, "not found")

```

OUTPUT:

```

Charan found
Esha not found
Bhavya found

```

JUSTIFICATION:

Improving Loop and Conditional Performance

Using a **HashSet** instead of nested loops improves lookup efficiency. This reduces time complexity and enhances the overall performance of the program.

TASK4:

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

- Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

- Detect hardcoded numeric values used directly in expressions.
- Define descriptive constants at the beginning of the program.
- Update calculations to use these constants without changing results.

Sample Code:

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
```

```
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations..

PROMPT:

Identify hardcoded numerical values used in the Python program and replace them with descriptive constants. Update the program so that the calculations remain the same but the code becomes more readable and maintainable.

ORIGINAL CODE:

```
lab_14_5.py
1  print("Simple Interest:", (1000 * 2 * 5) / 100)
2  print("Simple Interest:", (2000 * 2 * 5) / 100)
```

CODE:


```
Simple Interest: 100.0
Simple Interest: 200.0
```

REFACTORED CODE:

```
lab_14_5.py > ...
1  RATE = 2
2  TIME = 5
3  DIVISOR = 100
4
5  def calculate_interest(principal):
6      """Calculate simple interest."""
7      return (principal * RATE * TIME) / DIVISOR
8
9  print("Simple Interest:", calculate_interest(1000))
10 print("Simple Interest:", calculate_interest(2000))
```

CODE:

```
Simple Interest: 100.0
Simple Interest: 200.0
```

JUSTIFICATION:

Replacing Hardcoded Values with Constants

Replacing hardcoded values with named constants improves readability and maintainability. It also allows easy modification of values without changing multiple parts of the code.

TASK5:

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

- Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..

Instructions:

- Replace non-descriptive variable names with meaningful identifiers.
- Add inline comments where the logic may be unclear.
- Ensure program output remains the same.

Sample Code:

```
x = 50
y = 30
z = x + y
print(z)
```

Expected Output:

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

PROMPT:

Improve the Python code readability by renaming unclear variables and adding meaningful comments without changing the program functionality.

ORIGINAL CODE:

```
lab_14_5.py > ...
1  x = 50
2  y = 30
3  z = x + y
4  print(z)
```

OUTPUT:

```
80
```

REFACTORED CODE:

```
lab_14_5.py > ...  
1  # Store two numbers  
2  first_number = 50  
3  second_number = 30  
4  
5  # Calculate their sum  
6  total_sum = first_number + second_number  
7  |  
8  # Print the result  
9  print(total_sum)
```

OUTPUT:

```
80
```

JUSTIFICATION:

Enhancing Variable Naming and Code Clarity

Using meaningful variable names and adding comments improves code readability. This helps developers understand the program logic quickly and reduces confusion during maintenance.